COMP 211 · CHECKOFF 3 · TOPIC: SECURITY

I broke into my own program.

Stack Buffer Overflows: how one extra letter can beat a password check, and why C lets it happen

Shlok Bhutani

Buffer Overflows and Memory Vulnerabilities → Stack Buffer Overflows

WHY I CHOSE THIS TOPIC

Can one extra letter break a password check?

It’s hacking

I wanted to see how attackers actually break programs, not just hear that C is “unsafe.”

It builds on 211

The stack, arrays, strings and ASCII. Everything we’ve already done this semester.

It still happens

In 2026 the same bug let researchers take over office phones.

Today: what it is → a live break-in → a real 2026 attack → how to stop it

What’s a buffer overflow?

Putting more data into a box than it can hold, so the extra **spills onto whatever is next to it**.

Buffer

An array. A fixed-size box in memory, like `char name[8]`.

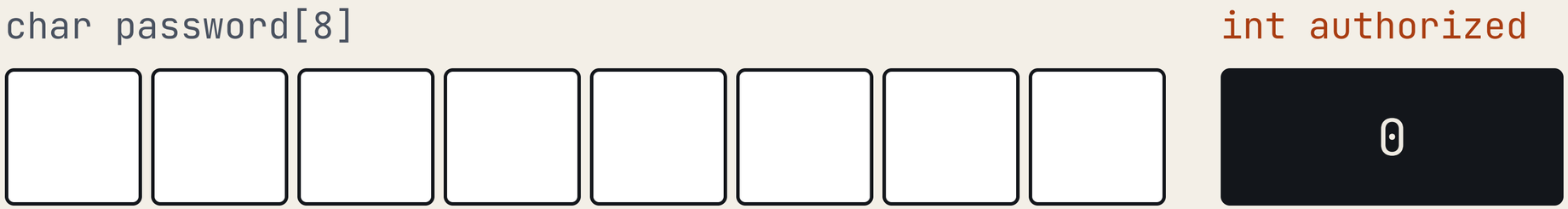
Overflow

Writing past the end of that box.

Stack

Where a function’s local variables live while it runs.

Variables sit right next to each other



My two variables

8 slots for the password. Right after it, a number that decides if you get in. **0 = locked out. Anything else = you're in.**

Where they live

On the stack. Each function call gets its own chunk for its local variables, packed close together.

C never checks if it fits

Python

```
box = [0] * 8
box[20] = 1
```

IndexError. Python stops you.

C

```
char box[8];
box[20] = 1;
```

No error. It just writes there.

Why? Checking every array access costs time. C chose speed and trusts you to get the size right.

The program

```
struct account {
char password[8];
int authorized; // 0 = no
};

scanf("%s", user.password);
// ↑ no limit on how much it reads

if (strcmp(user.password, "secret") == 0)
user.authorized = 1;

if (user.authorized) printf("Access granted!");
```

What it should do

1. Ask for a password
2. Compare it to **secret**
3. Match? Set authorized to 1
4. authorized isn't 0? Let them in

So the only way in is to know the password. Right?

LIVE DEMO · YOUR GUESS

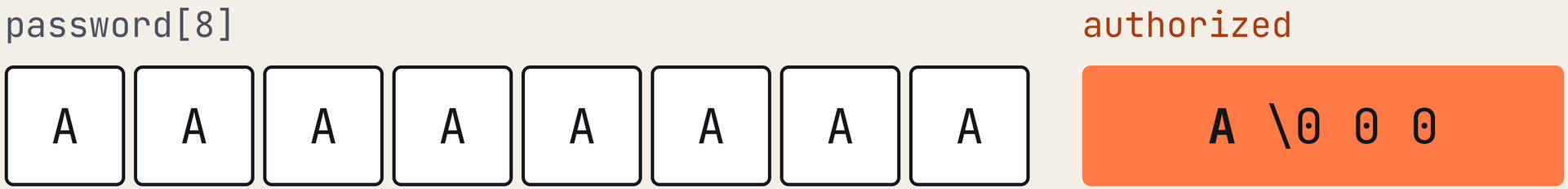
`secret` → Access granted

`AAAAAAAA` → 8 letters → Access denied

`AAAAAAAAA` → 9 letters → ???

What happens with one more letter?

The 9th letter spilled over



`authorized = 65`

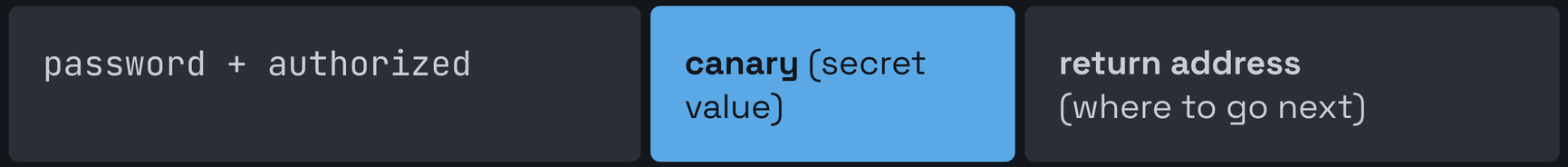
65 is the ASCII code for **A**. In C, anything that isn't 0 counts as true.

What I actually got

I typed	authorized	Result
`secret`	`1`	Granted (correct)
`hello`	`0`	Denied (correct)
`A × 8`	`0`	Denied
`A × 9`	`65`	**Granted. Broken in.**
`A × 12`	`1094795585`	**Granted. Broken in.**
`A × 60`	—	**Program crashed?!**

The compiler fought back

```
*** stack smashing detected ***: terminated
```



The compiler hides a secret value on the stack. If it changes, something overflowed, so the program stops on purpose.

Behind it is the **return address**. Overwrite that and you control what code runs next. That's what real attackers go for.

REAL LIFE · 2026

Hacking an office desk phone

What happened

Jan 6: a Rapid7 researcher reports a bug in Grandstream GXP1600 office phones.

Early Feb: Grandstream releases a firmware fix.

Feb 18: the bug goes public, rated 9.3 out of 10.

The bug

The phone's web API stored incoming data on the stack **without checking its length**. Same mistake as my demo.

What an attacker could do

With no password: run any code as root and listen in on calls.

CVE-2026-2329 · Rapid7, The Hacker News, SecurityWeek

How to actually fix it

1. Tell C the size

```
fgets(buf,
sizeof(buf), stdin);
```

Extra letters get ignored. I re-ran my 9 A's: denied.

2. Keep protections on

Canaries and compiler warnings are on by default. Don't turn them off.

3. Use a safer language

Python, Java and Rust check sizes for you. The Linux kernel now accepts Rust partly for this reason.

And for devices like those phones: **install the update.**

TAKEAWAY

C trusts you completely. That's its power and its danger.

What surprised me

One extra letter was enough. And about 70% of Microsoft's security bugs are memory bugs like this.

What I still want to know

How do real attackers get past canaries?

Questions?

Try breaking it yourself: the code, slides and sources are on GitHub.

`github.com/shlokbhutani13/comp211-checkoff3-buffer-overflow`

Shlok Bhutani · COMP 211 · Checkoff 3